

KONGU ENGINEERING COLLEGE
(AUTONOMOUS)
PERUNDURAI, ERODE – 638 060

PROJECT REPORT

EXPERIMENT - 1

PROJECT TITLE

**SMART REFRIGERATOR MONITORING SYSTEM USING
DHT11 SENSOR**

SUBMITTED BY

ABISHEK R S - 24MER002

GOWRISANKAR K - 24MER010

NAVEENRAJ K - 24MER035

PRAVEEN N M - 24MER043

Design and simulate a Smart Refrigerator Monitoring System using DHT11 Sensor

1. Project review :

The **Smart Refrigerator Monitoring System Using DHT11 Sensor** is a compact prototype designed to monitor the **temperature and humidity inside a refrigerator**. The system uses an **ESP8266 NodeMCU** as the central controller and a **DHT11 sensor** to measure the internal environmental conditions.

The DHT11 continuously sends temperature and humidity readings to the ESP8266. The controller processes these values and compares the temperature with a predefined limit. When the temperature exceeds the selected limit, a **buzzer provides an audible warning**, and a **relay module can be used to control a suitable cooling load**.

The proposed system provides a **simple, low-cost, and automatic method of refrigerator monitoring**. It can also be further developed with Wi-Fi, cloud monitoring, mobile notifications, and data logging for a complete IoT-based smart refrigerator system.

2. Problem Statement :

Conventional refrigerators may not provide **continuous monitoring of temperature and humidity** inside the storage compartment. Users may not know when the internal temperature rises beyond a suitable level, which can affect food freshness and increase the possibility of food spoilage.

Another challenge is that **manual temperature checking** is inconvenient and does not provide an immediate warning when abnormal conditions occur. There is also a need for a simple and low-cost system that can automatically monitor refrigerator conditions.

The proposed system addresses these problems by integrating a **DHT11 sensor, ESP8266 NodeMCU, buzzer, and relay module**. The DHT11 measures temperature and humidity, while the ESP8266 processes the sensor readings. When the temperature exceeds the predefined limit, the buzzer provides an alert and the relay can be used for suitable cooling control.

3. Proposed Solution :

The proposed solution consists of an **ESP8266-based Smart Refrigerator Monitoring System**. The **DHT11 sensor** is used to measure the temperature and humidity inside the refrigerator and sends the readings to the ESP8266 NodeMCU.

The ESP8266 processes the sensor readings and compares the measured temperature with a **predefined temperature limit**. If the temperature exceeds the selected limit, the ESP8266 activates the **buzzer** to provide an audible warning. A **relay module** can also be activated to control a suitable cooling load.

Proposed Operation:

1. Start the system and initialize the ESP8266.
2. Initialize the DHT11 sensor, relay, and buzzer.
3. DHT11 measures the temperature and humidity.
4. DHT11 sends the sensor data to the ESP8266.
5. ESP8266 processes the temperature and humidity readings.
6. The temperature is compared with the predefined limit.
7. If the temperature is above the limit, the buzzer is activated.
8. The relay can be activated for suitable cooling control.
9. If the temperature is normal, the buzzer remains OFF.
10. The system continuously repeats the monitoring process.

4. System Architecture :

The **Smart Refrigerator Monitoring System** consists of five main sections:

Input Section:

The input section consists of:

- **DHT11 Sensor** – Measures temperature and humidity inside the refrigerator.
- The DHT11 sends the measured data to the ESP8266 NodeMCU.

Processing Section:

The **ESP8266 NodeMCU** acts as the central controller.

- Receives temperature and humidity data from the DHT11.
- Processes the sensor readings.
- Compares the temperature with the predefined limit.
- Controls the buzzer and relay according to the programmed condition.

Actuation Section:

The **Relay Module** is used for cooling control.

- Relay receives a control signal from the ESP8266.
- It can control a suitable cooling load when the temperature becomes high.

Indication Section:

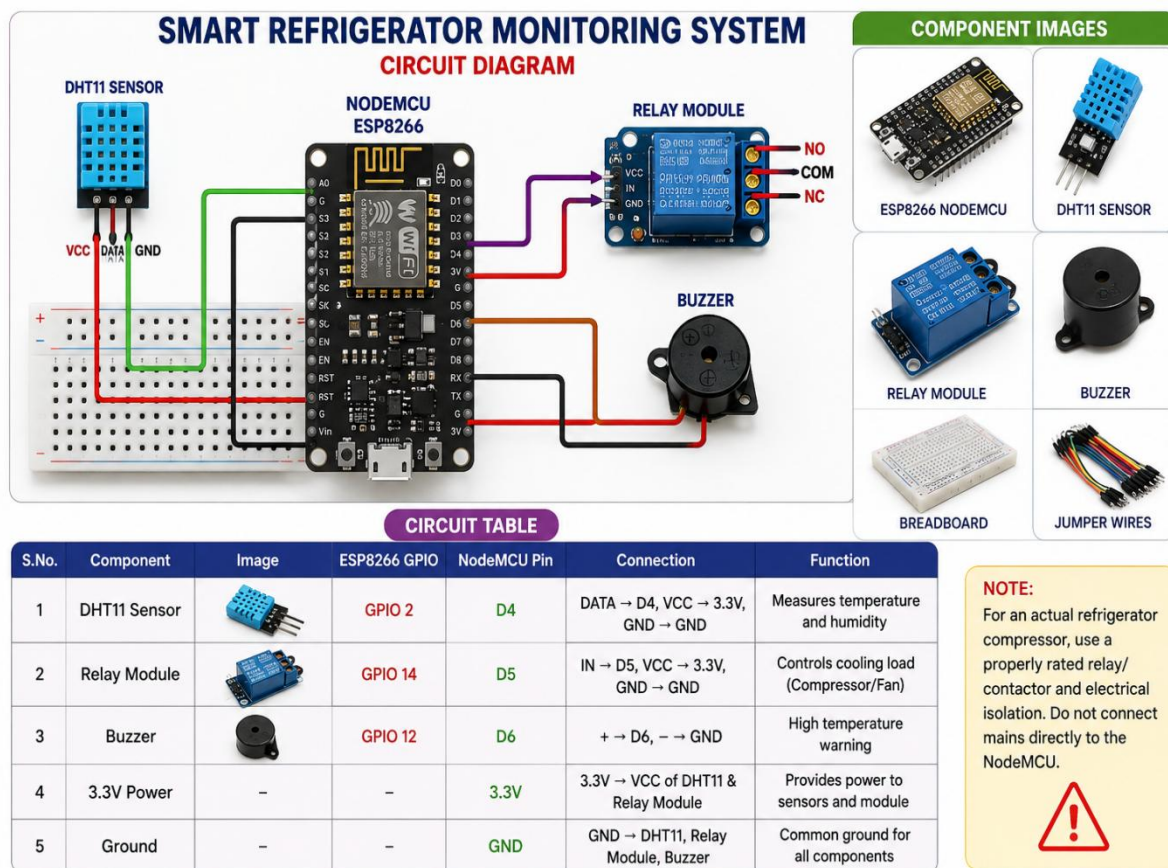
The **Buzzer** provides an audible warning.

- Buzzer OFF → Normal temperature condition.
- Buzzer ON → Temperature exceeds the predefined limit.

Connectivity Section:

The **ESP8266** has built-in Wi-Fi capability.

5. CIRCUIT TABLE :



Important Pin Mapping:

GPIO 2 → D4 → DHT11 Sensor

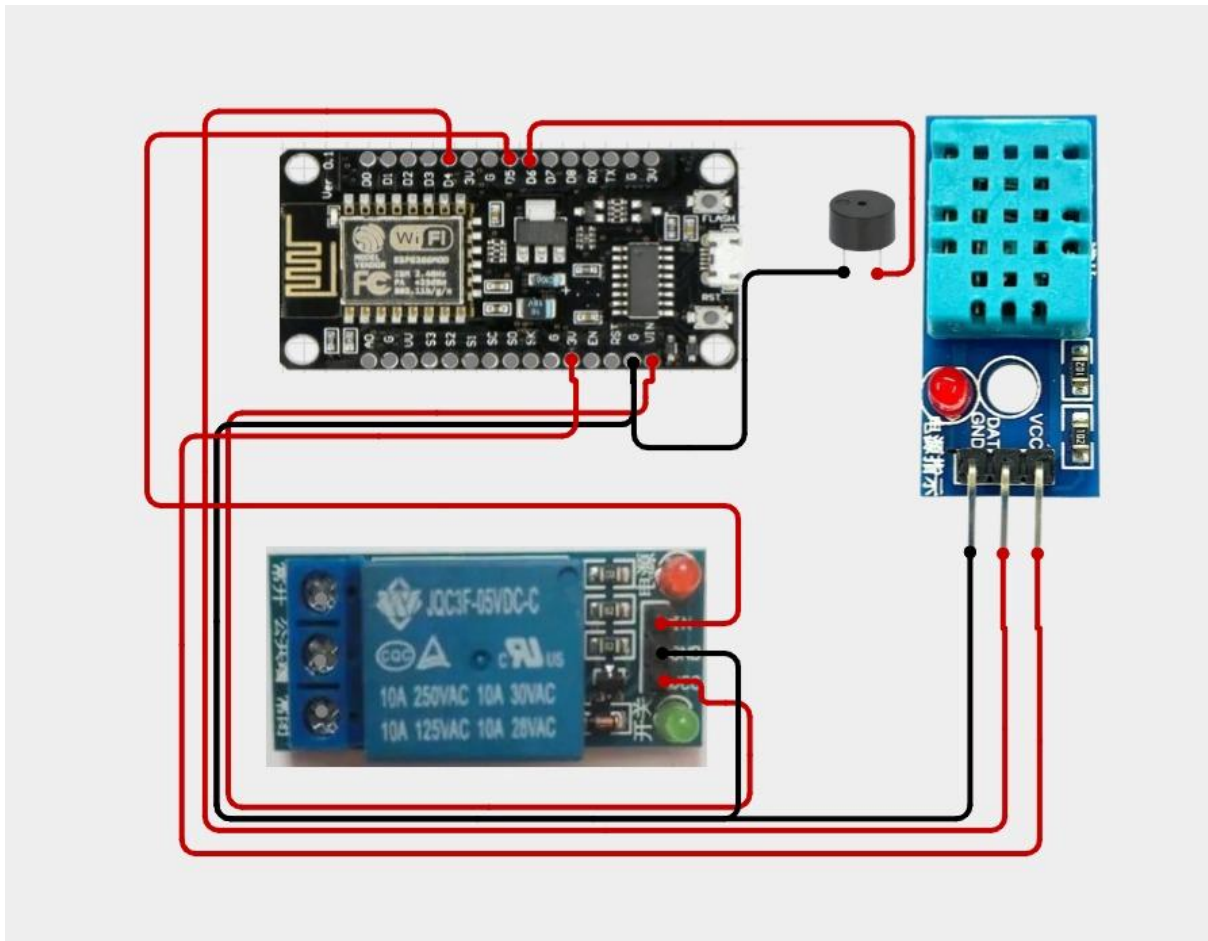
GPIO 14 → D5 → Relay Module

GPIO 12 → D6 → Buzzer

3.3V → DHT11 VCC → Sensor Power

GND → DHT11 GND → Common Ground

6. Circuit Design :



8.Coding

```
#include <DHT.h>
```

```
// =====
```

```
// SMART REFRIGERATOR MONITORING SYSTEM
```

```
// ESP8266 NodeMCU + DHT11 + Relay + Buzzer
```

```
// =====
```

```
// DHT11 DATA → D4 / GPIO2
```

```
#define DHTPIN 2
```

```
#define DHTTYPE DHT11
```

```
// Relay IN → D5 / GPIO14
```

```
#define RELAY_PIN 14
```

```
// Buzzer + → D6 / GPIO12
```

```
#define BUZZER_PIN 12
```

```
// Create DHT object
```

```
DHT dht(DHTPIN, DHTTYPE);
```

```
// =====  
// SETUP  
// =====  
  
void setup()  
{  
  // Start Serial Monitor  
  Serial.begin(9600);  
  
  // Start DHT11  
  dht.begin();  
  
  // Set relay and buzzer as outputs  
  pinMode(RELAY_PIN, OUTPUT);  
  pinMode(BUZZER_PIN, OUTPUT);  
  
  // Initial state  
  digitalWrite(RELAY_PIN, LOW);  
  digitalWrite(BUZZER_PIN, LOW);  
  
  // Startup message  
  Serial.println();  
  Serial.println("=====");  
  Serial.println(" SMART REFRIGERATOR MONITORING");  
  Serial.println("=====");
```



```
Serial.println("System Started");
Serial.println();
}

// =====

// LOOP

// =====

void loop()
{
    // Read temperature
    float temperature = dht.readTemperature();

    // Read humidity
    float humidity = dht.readHumidity();

    // =====

    // CHECK DHT11 SENSOR

    // =====

    if (isnan(temperature) || isnan(humidity))
    {
        Serial.println("ERROR: DHT11 sensor not responding!");
        Serial.println("Check VCC, DATA and GND connections.");
    }
}
```

```
// Turn buzzer ON
digitalWrite(BUZZER_PIN, HIGH);

delay(2000);

return;
}

// =====
// DISPLAY TEMPERATURE AND HUMIDITY
// =====

Serial.println("-----");

Serial.print("Temperature : ");
Serial.print(temperature);
Serial.println(" C");

Serial.print("Humidity  :");
Serial.print(humidity);
Serial.println(" %");

// =====
```

```
// RELAY CONTROL

// =====

if (temperature > 5.0)
{
    digitalWrite(RELAY_PIN, HIGH);

    Serial.println("Relay    : ON");
    Serial.println("Cooling   : REQUIRED");
}
else
{
    digitalWrite(RELAY_PIN, LOW);

    Serial.println("Relay    : OFF");
    Serial.println("Cooling   : NORMAL");
}

// =====

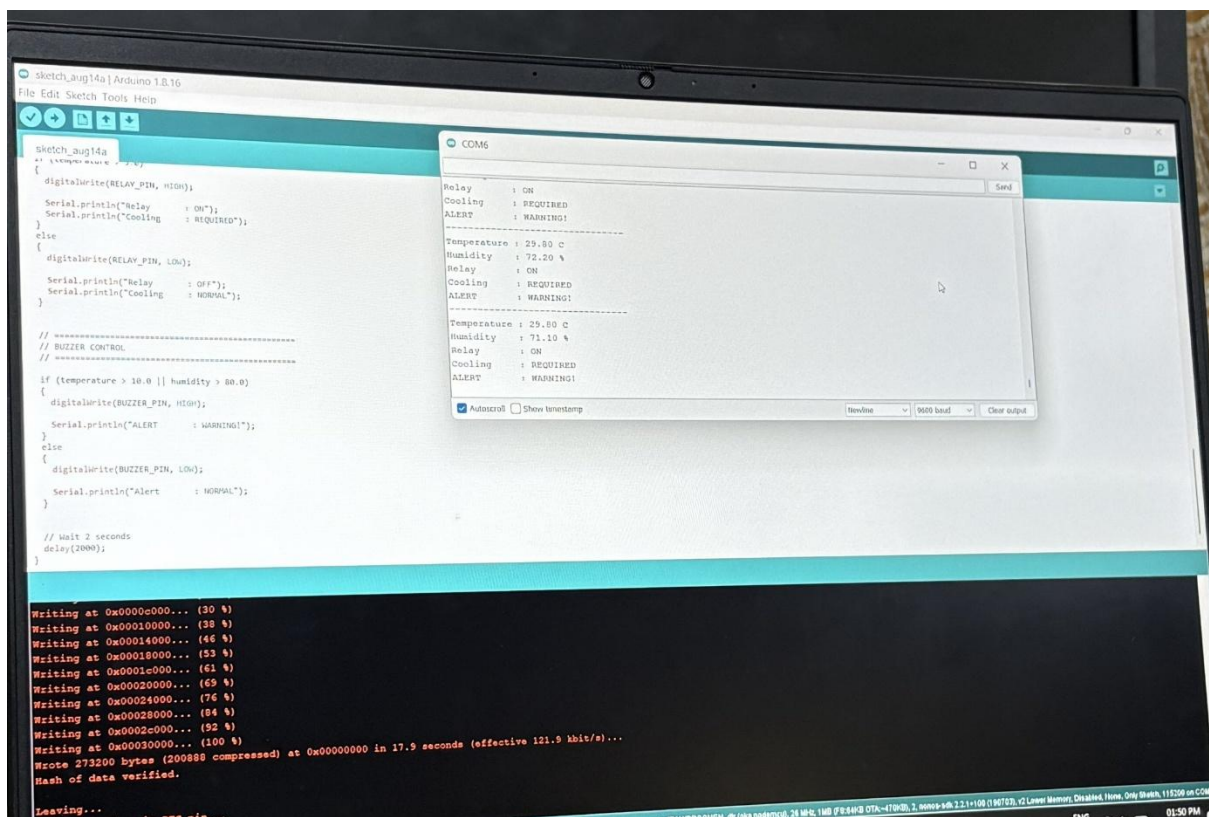
// BUZZER CONTROL

// =====

if (temperature > 10.0 || humidity > 80.0)
{
    digitalWrite(BUZZER_PIN, HIGH);
```

```
    Serial.println("ALERT    : WARNING!");  
}  
else  
{  
    digitalWrite(BUZZER_PIN, LOW);  
  
    Serial.println("Alert    : NORMAL");  
}  
  
// Wait 2 seconds  
delay(2000);  
}
```

CODING OUTPUT:



```
sketch_aug14a | Arduino 1.8.16
File Edit Sketch Tools Help

sketch_aug14a
// temperature sensor
{
  digitalWrite(RELAY_PIN, HIGH);
  Serial.println("Relay : ON");
  Serial.println("Cooling : REQUIRED");
}
else
{
  digitalWrite(RELAY_PIN, LOW);
  Serial.println("Relay : OFF");
  Serial.println("Cooling : NORMAL");
}

// =====
// BUZZER CONTROL
// =====
if (temperature > 30.0 || humidity > 80.0)
{
  digitalWrite(BUZZER_PIN, HIGH);
  Serial.println("ALERT : WARNING!");
}
else
{
  digitalWrite(BUZZER_PIN, LOW);
  Serial.println("Alert : NORMAL");
}

// Wait 2 seconds
delay(2000);
}

Writing at 0x0000c000... (30 %)
Writing at 0x00010000... (38 %)
Writing at 0x00014000... (46 %)
Writing at 0x00018000... (53 %)
Writing at 0x0001c000... (61 %)
Writing at 0x00020000... (69 %)
Writing at 0x00024000... (76 %)
Writing at 0x00028000... (84 %)
Writing at 0x0002c000... (92 %)
Writing at 0x00030000... (100 %)
Wrote 273200 bytes (200888 compressed) at 0x00000000 in 17.9 seconds (effective 121.9 kbit/s)...
Hash of data verified.
Leaving...
```

```
COM6
Relay : ON
Cooling : REQUIRED
ALERT : WARNING!
-----
Temperature : 29.80 c
Humidity : 72.20 %
Relay : ON
Cooling : REQUIRED
ALERT : WARNING!
-----
Temperature : 29.80 C
Humidity : 71.10 %
Relay : ON
Cooling : REQUIRED
ALERT : WARNING!

Autoscroll Show timestamp Newline 9600 baud Clear output
```

9. System Working :

- **Power ON the System**

- The ESP8266 NodeMCU is powered ON.

- **Initialize Components**

- The DHT11 sensor, relay module, and buzzer are initialized.

- **Measure Temperature**

- The DHT11 sensor measures the temperature inside the refrigerator.

- **Measure Humidity**

- The DHT11 sensor measures the humidity level inside the refrigerator.

- **Send Sensor Data**

- The DHT11 sends temperature and humidity data to the ESP8266 through **D4 (GPIO2)**.

- **Process the Data**

- The ESP8266 receives and processes the sensor readings.

- **Check Temperature**

- The measured temperature is compared with the predefined temperature limit.

- **High Temperature Condition**

- If the temperature is above the limit:
 - **Buzzer → ON**
 - **Relay → ON**
 - Cooling control is activated.
- **Normal Temperature Condition**
 - If the temperature is within the limit:
 - **Buzzer → OFF**
 - Relay remains in its normal state.
- **Repeat the Process**
 - The DHT11 takes new readings and the ESP8266 continuously monitors the refrigerator conditions.

10. PROTOTYPE IMPLEMENTATION :

The prototype is implemented using an **ESP8266 NodeMCU, DHT11 sensor, relay module, buzzer, breadboard, and jumper wires**. The DHT11 sensor is connected to the ESP8266 to monitor the temperature and humidity inside the refrigerator.

Hardware Setup

1. Connect the **DHT11 DATA pin → D4 (GPIO2)** of ESP8266.
2. Connect the **Relay IN pin → D5 (GPIO14)**.
3. Connect the **Buzzer → D6 (GPIO12)**.
4. Connect the DHT11 **VCC → 3.3V** and **GND → GND**.
5. Connect the relay module according to its required power supply.
6. Connect all required low-voltage components using jumper wires.
7. Upload the program to the ESP8266 using the Arduino IDE.
8. Power ON the prototype and observe the temperature and humidity readings.
9. When the temperature exceeds the predefined limit, the **buzzer and relay are activated**.
10. The system continuously monitors the refrigerator conditions.

Prototype Operation

The prototype demonstrates automatic temperature monitoring and warning. The **DHT11 sensor** provides the environmental data, while the **ESP8266 processes the readings** and controls the buzzer and relay based on the programmed condition.

Prototype Working :

- The **ESP8266 NodeMCU** is powered ON and initializes all connected components.
- The **DHT11 sensor** measures the temperature and humidity inside the refrigerator.
- The sensor sends the measured values to the **ESP8266 through D4 (GPIO2)**.
- The ESP8266 processes the temperature and humidity readings.
- The measured temperature is compared with the **predefined temperature limit**.
- If the temperature is **above the set limit**, the **buzzer turns ON** to alert the user.
- At the same time, the **relay connected to D5 (GPIO14)** is activated for suitable cooling-load control.
- If the temperature is **within the normal limit**, the buzzer remains OFF and the relay stays in its normal state.
- The system waits for a short interval and reads the DHT11 sensor again.
- This process continues **automatically and continuously**, providing real-time monitoring of the refrigerator conditions.

11. Bill of Materials :

S. No.	Component	Specification	Quantity
1	ESP8266 NodeMCU	Wi-Fi Development Board	1
2	DHT11 Sensor	Temperature & Humidity Sensor	1
3	Relay Module	1-Channel Relay	1
4	Buzzer	3.3V/5V Buzzer	1
5	Breadboard	Standard Prototype Board	1
6	Jumper Wires	Male-Male / Male-Female	1 Set
7	USB Cable	Micro-USB	1
8	Power Supply	Suitable regulated supply	1

12. Results & Performance Analysis :

- The **DHT11 sensor** successfully measures temperature and humidity inside the refrigerator.
- The **ESP8266 NodeMCU** successfully receives and processes the sensor readings.
- The system provides **continuous monitoring** of refrigerator conditions.
- When the temperature exceeds the predefined limit, the **buzzer is activated**.
- The **relay is activated** to demonstrate cooling-load control.
- When the temperature returns to the normal range, the **buzzer turns OFF**.
- The system provides an **automatic and reliable monitoring process**.

Results

1. The DHT11 sensor successfully measures the **temperature and humidity** inside the refrigerator.
2. The ESP8266 NodeMCU successfully **receives and processes the sensor data**.
3. The system continuously monitors the refrigerator conditions.
4. When the temperature exceeds the predefined limit, the **buzzer turns ON**.
5. The **relay activates** to demonstrate cooling-load control.
6. When the temperature returns to the normal range, the **buzzer turns OFF**.
7. The system provides **automatic and continuous refrigerator monitoring**.
8. The prototype demonstrates the basic operation of a **smart refrigerator monitoring system**.

Performance Analysis

Parameter	Expected Performance
Temperature Monitoring	Continuous
Humidity Monitoring	Continuous
Sensor Response	Real-time reading
High Temperature Alert	Buzzer ON
Cooling Control	Relay ON
Normal Condition	Buzzer OFF